

协同过滤（Collaborative Filtering）第一章 完全总结

这是推荐系统最核心的一章，通俗来说就是：“和你口味相似的人喜欢什么，我就推荐给你什么”。

一、什么是协同过滤？

1.1 核心思想

协同过滤（CF）是一种"借群体智慧"的推荐方法。基本假设很简单：

过去有相似偏好的用户，未来也会有相似的偏好。

举例：如果你和小明都看了很多相同的电影，给了差不多的高分，那系统就认为你俩口味相似。当小明看了你没看过的新电影，系统就把它推荐给你。

1.2 纯协同过滤的输入与输出

	内容
输入	只给一个用户-物品评分矩阵（例如用户对电影的1-5星评分），不依赖物品内容、用户资料等附加信息
输出	① 数值预测（预测你对某物品的评分）；② Top-N 推荐列表（给你推荐N个可能喜欢的物品）

二、两大基本路线：User-CF vs Item-CF

这是协同过滤最核心的两条路，贯穿整个章节。

2.1 基于用户的协同过滤（User-based CF）

思路：找和你评分习惯相似的"邻居"用户，用他们的评分来预测你的。

步骤：

- 给你（Alice）推荐她没看过的 Item5
- 找到已经对 Item5 评过分的用户（User1-4）
- 筛选出和 Alice 口味最相似的邻居
- 用邻居的评分加权平均来预测 Alice 的评分
- 推荐评分最高的物品

```
Alice:  5  3  4  4  ?    ← 问号是待预测的
User1:  3  1  2  3  3
User2:  4  3  4  3  5
User3:  3  3  1  5  4
User4:  1  5  5  2  1
```

2.2 基于物品的协同过滤（Item-based CF）

思路反转：不是找相似用户，而是找和你喜欢的物品相似的其他物品。

你喜欢 Item1、Item2、Item3，那我就找和这些物品相似的 Item5，用你对 Item1/2/3 的评分来预测 Item5。

优势：

- 用户量远大于物品量时，计算量更小（亿级用户 vs 千万级商品）
- 物品之间的相似度比用户相似度更稳定（人的口味会漂移，但"爱情片"和"爱情片"的相似性不会）
- 能缓解新用户冷启动问题

2.3 两者对比

维度	User-CF	Item-CF
找什么相似	用户-用户	物品-物品
稳定性	用户口味会变，相似度不稳定	物品属性稳定，相似度稳定
计算复杂度	用户多时开销大	物品少时更高效
预处理	可预计算用户相似度	可预计算物品相似度（亚马逊2003年方案）

三、相似度度量——核心中的核心

做 CF 最关键的三个问题：① 怎么量"相似"？② 用几个邻居？③ 怎么加权预测？

3.1 皮尔逊相关系数（Pearson Correlation）★

这是 User-CF 中最常用的相似度指标。

为什么不用余弦相似度直接用？

想象两个用户 A 和 B：

- A 评分很严格，1-3分居多
- B 评分很宽松，4-5分居多

如果直接用余弦，会因为评分"档位"不同而判断为不相似。皮尔逊先减去各自均值（中心化），只看趋势是否一致——这就好比把不同人的"刻度尺"校准了。

$$\text{sim}(a,b) = \frac{\sum(r_{a,p} - \bar{r}_a)(r_{b,p} - \bar{r}_b)}{\sqrt{[\sum(r_{a,p} - \bar{r}_a)^2 \cdot \sum(r_{b,p} - \bar{r}_b)^2]}}$$

- 取值范围：[-1, 1], 1 表示完全正相关, -1 表示完全负相关
- 在上例中: Alice vs User1 sim=0.85, Alice vs User4 sim=-0.79

3.2 调整余弦相似度 (Adjusted Cosine)

用于 Item-CF 的场景。思路和皮尔逊类似，但减的是用户的平均分：

$$\text{sim}(a,b) = \frac{\sum(r_{u,a} - \bar{r}_u)(r_{u,b} - \bar{r}_u)}{\sqrt{[\sum(r_{u,a} - \bar{r}_u)^2 \cdot \sum(r_{u,b} - \bar{r}_u)^2]}}$$

四、如何做预测？

4.1 预测公式 (User-CF)

拿到邻居后，用加权平均的方式预测：

$$\text{pred}(a,p) = \bar{r}_a + \frac{\sum(\text{sim}_{a,\beta} \cdot (r_{\beta,p} - \bar{r}_\beta))}{\sum|\text{sim}_{a,\beta}|}$$

翻译成人话：

你的预测评分 = 你的平均分 + 你邻居们评分的"偏差"按相似度加权求和

4.2 预测公式 (Item-CF)

$$\text{pred}(u,p) = \frac{\sum(\text{sim}(i,p) \cdot r_{u,i})}{\sum|\text{sim}(i,p)|}$$

翻译：用你评过的那些物品，与目标物品的相似度做加权平均。

4.3 邻居数量的选择

研究表明：选择20到50个邻居是比较合理的 (Herlocker et al. 2002) 。

五、相似度的工程改进

5.1 方差加权

在有争议的物品上邻居的一致性更有意义 ("大家都不喜欢"的一致性 ≠ "大家都喜欢"的一致性)。给方差大的物品更高权重。

5.2 重要性加权 (共同评分数量)

两个用户只共同评了1个物品，相似度可能是偶然的；共同评了50个物品，相似度更可靠。共同评分越少，权重越低。

5.3 案例放大 (Case Amplification)

对相似度值接近1的"铁杆邻居"放大其权重，让非常相似的邻居更有话语权。

六、评分方式：显式 vs 隐式

6.1 显式评分

用户主动打分（1-5星、1-10分等）。**最准确但用户懒得给。**

关键问题：评分粒度（10分制比5分制更细腻），如何激励用户评分（积分、徽章、社交）。

6.2 隐式评分

用户不主动打分，而是系统从行为中推断：购买、点击、浏览时长、下载等。**收集成本低但解释有歧义**（买书可能送人，点开可能秒退）。

两者可以结合使用，但隐式评分需要谨慎解读。

七、数据稀疏性——协同过滤的死穴

7.1 冷启动问题

新用户没评过任何分，系统完全没有参考依据 → 无法推荐。

解决思路：

- 要求初始评分（注册时勾选兴趣类别）
- 用其他方法过渡：基于内容推荐（物品特征）、基于人口统计（年龄职业）、非个性化（热榜）
- 用更好的算法扩大邻居

7.2 递归协同过滤 (Recursive CF)

如果邻居 n 本身也没给目标物品评分怎么办？→ **递归**！先用这个邻居的邻居来预测 n 的评分，再用这个预测值参与最终预测。

Alice → User1 → User2 (有评分) → 回推

7.3 稀疏度示例

MovieLens-100K 数据集稀疏度约 **6.3%**：

- 943用户 × 1682电影 = 158万可能的评分
- 实际只有10万条评分
- 平均每用户只评了约106部电影

八、基于模型的方法（Model-based CF）

核心思路：把"离线计算"做充分，换取"在线预测"的速度和准确性。

8.1 矩阵分解 / SVD

这是本章最重要的进阶知识点。

核心思想：把用户-物品评分矩阵 M 分解为三个矩阵的乘积：

$$M \approx U_k \cdot \Sigma_k \cdot V_k^T$$

- U_k ：每行是一个用户在 k 维"潜在因子空间"的位置
- V_k^T ：每列是一个物品在 k 维空间的位置
- Σ_k ：对角矩阵，每个值代表对应因子（维度）的重要性

为什么能降维？

- 原始矩阵可能充满噪声（偶然的评分偏差）
- 取前 k 个最大奇异值（ $k=20\sim100$ ），忽略次要维度 → 保留核心信号，过滤噪声
- "细节越少越好"：模糊的红黑花朵图比清晰的更能看出和鲜红花朵的相似性

降维后的优势：

- 用户和物品都被投影到同一个 k 维空间
- 预测 = 用户向量 $\times$ 物品向量（点积），**极快**
- 线下分解完成，线上毫秒级响应

8.2 关联规则挖掘

从购物行为中挖掘规则，例如：

"买啤酒的用户70%也会买尿布"

两个核心指标：

- **支持度 (support)**：两个商品一起被买的交易 / 总交易 → 规则"普遍吗？"
- **置信度 (confidence)**：买了前项的商品中，也买了后项的比例 → 规则"靠谱吗？"

应用到推荐：把5分制评分转成0/1（1=高于平均），挖掘"买了X → 可能也喜欢Y"的规则。

8.3 概率方法

用贝叶斯定理计算用户给某物品打某个评分的概率：

$$P(\text{Item5=评分} \mid \text{用户历史评分}) \propto P(\text{用户历史评分} \mid \text{Item5=评分}) \times P(\text{Item5=评分})$$

假设各评分相互独立，连乘各项概率，取概率最大的评分作为预测。

问题：如果某评分组合在历史数据中从未出现 → 概率为0（零概率问题），需要平滑技术处理。

8.4 Slope One 预测器

极其简单快速的预测器，基本思想：

用户对物品的偏好差异是稳定的

公式： $p(\text{Alice}, \text{Item5}) = \text{Alice对Item1的评分} + (\text{Item5平均分} - \text{Item1平均分})$

或者更一般： $f(x) = x + b$ ，找"斜率为1"的线性函数来拟合评分差异。

适合追求简单、快速、效果好的场景。

8.5 RF-Rec (Rating Frequency) 预测器

利用评分频率来做预测：

- 用户评分频率：反映用户倾向于给什么评分
- 物品评分频率：反映物品倾向于收到什么评分

预测公式： $p(u, i) = \text{argmax}_v [f_user(u, v) \times f_item(i, v)]$

即：找用户和物品共同最频繁出现的评分值。计算极其高效。

九、Netflix竞赛与融合模型

9.1 Netflix Prize 背景

- 2006年，Netflix悬赏100万美元
- 要求：比Cinematch系统RMSE提升10%
- 获奖方案 Yehuda Koren (2008)：把邻域模型 + 潜在因子模型融合

9.2 Koren 的融合预测函数

$$\hat{r}(u, i) = \mu + b_u + b_i + p_u^T \cdot q_i$$

其中：

- μ = 全局平均评分
- b_u = 用户偏差（你比平均多给/少给多少分）
- b_i = 物品偏差（该物品比平均好/差多少）
- $p_u^T \cdot q_i$ = 用户和物品在潜在因子空间的点积

训练目标：最小化"预测误差的平方 + 正则化惩罚项"（避免过拟合）

两种模型的分工：

- 潜在因子模型：捕捉整体弱信号（全局趋势）
- 邻域模型：捕捉局部强关联（相邻物品之间的偏好）

十、评估指标

指标	含义	侧重点
MAE	平均绝对误差，预测评分与实际评分偏差的平均值	线性惩罚误差
RMSE	均方根误差，对大偏差更敏感	惩罚极端错误
Precision@K	推荐列表中有多少是相关的	推荐准确性
Recall@K	相关物品中有多少被推荐了	推荐覆盖率
MAP / NDCG	综合考虑排序质量的指标	推荐列表排序质量

十一、知识全景图

协同过滤 (CF)

├ 基于邻域 (Memory-based)

├ User-based CF

└ Item-based CF

├ 相似度: Pearson / Cosine / 调整余弦

└ 预测: 加权平均

├ 基于模型 (Model-based)

├ 矩阵分解 / SVD (降维)

├ 关联规则

├ 概率模型 (贝叶斯)

├ Slope One (简单线性)

└ RF-Rec (频率统计)

└ 融合模型 (邻域 + 潜在因子)

十二、真题级要点速记

- CF的核心 = 找相似 + 加权预测，User-CF 找相似用户，Item-CF 找相似物品。
- 皮尔逊相关系数是相似度的黄金标准，因为它去除了用户评分"刻度"差异。
- Item-CF 的最大优势是物品相似度比用户相似度更稳定。
- SVD 降维的核心价值：用离线复杂计算换在线毫秒预测，同时去噪。
- 稀疏性是 CF 的最大挑战，冷启动用其他方法过渡，递归CF扩大邻居。
- Koren融合模型 = 全局平均 + 用户偏差 + 物品偏差 + 潜在因子点积。
- 没有最好的 CF 方法，不同方法在不同场景下各有优劣，评估用 RMSE/MAE，但也要关注意外性 (Serendipity) 。